

Matthew Hamilton

matthewhamilton3141@gmail.com | linkedin.com/in/matthewhamilton3141 | github.com/matthewhamilton3141 | matthewhamilton.dev

Education

University of Waterloo

Waterloo, ON

Bachelor of Applied Science in Systems Design Engineering

Sep. 2026 – May 2031 (Expected)

- **Scholarships:** Alumni @ Microsoft Engineering Scholarship (1 recipient/year, facultywide)
- **Relevant Coursework:** Digital Computation (SYDE 121, C++), Linear Algebra (MATH 115)

Projects

LaserTagNow | *Swift, SwiftUI, ARKit, NearbyInteraction, Network Framework*

2026

- Won \$1,200 + 2nd Place at SummerHacks
- Built a fully serverless, peer-to-peer multiplayer game for iOS supporting real-time multi-player matches with zero backend infrastructure, by designing a frame-independent networking protocol that encodes spatial events (hits, drops, positions) as affine combinations of player coordinates, letting each device resolve state into its own local AR frame without a shared world map
- Achieved reliable, real-time target-lock accuracy from noisy $\sim 5\text{Hz}$ Ultra-Wideband bearing data, by fusing NearbyInteraction ranging with ARKit camera pose estimation into a custom aim-assist algorithm balancing responsiveness against false locks (e.g., through walls)
- Shipped a complete real-time game loop — lobbies, combat, power-ups, death/respawn, and match summaries — across a 5,000+ line Swift/SwiftUI codebase, by architecting an idempotent, order-tolerant state-sync layer that kept UI stable despite late or dropped peer-to-peer messages

shield-in-alpasim | *Python, Docker, gRPC, CUDA, Omniverse, Hydra, NumPy, pytest, Linux*

2026

- Integrated an out-of-tree plugin into NVIDIA AlpaSim's Dockerized gRPC microservice stack — injecting the module + its dependency into the base container image with install-at-startup wiring, and registering it through AlpaSim's driver entry-point system
- Built a dependency-light, CPU-only core with 100+ unit tests and orchestrated end-to-end runs on cloud A100 GPUs via a reproducible, cost-aware workflow (Docker, Hydra configs, remote tmux); diagnosed cross-service integration bugs — a serialization/API mismatch and a bad coordinate transform — hardened with regression tests
- Designed a real-time per-step adapter bridging a trajectory-emitting ML policy and a per-step controller, keeping the hot path allocation-light in NumPy

Retermina | *Tauri v2, Rust, React, TypeScript*

2026 – Present

- Delivered a local-first, Claude-centric desktop terminal workspace by building on Tauri v2 (Rust + React/TypeScript), where a Rust backend spawns and multiplexes native PTY sessions over Tauri IPC with tools docked in a Cursor-style drawer
- Kept terminals — and the dev servers inside them — alive across app quit, crash, or auto-update by architecting a live-session layer that hosts every PTY in a separate re-exec'd process, reconnecting over a Unix-domain socket and replaying scrollbar on relaunch, with a grace-TTL reaper reaping lingering daemons
- Turned it into a supervision layer for coding agents: embedded Claude Code with a live context-window gauge (parses session JSONL to chart context fill against the model window), per-project token and cost accounting, and cross-restart session resume that reconnects the agent and rehydrates its transcript
- Shipped universal (Apple Silicon + Intel) macOS releases via a GitHub Actions pipeline and a minisign-signed in-app auto-updater that verifies each download against a `latest.json` manifest before applying

Technical Skills

Languages: TypeScript, Python, Rust, C++, JavaScript, HTML/CSS

Frameworks & Libraries: React, Node.js, Tauri, NumPy, PyTorch, OpenCV, stable-baselines3, Gymnasium

Tools & Platforms: Git, Linux, CUDA, TensorRT, ONNX, Docker, gRPC, Hydra, pytest, NVIDIA Container Toolkit, NGC, OpenUSD, NVIDIA Omniverse

AI / ML: 3D Gaussian Splatting, Depth Anything V2, reinforcement learning, visual SLAM, TensorRT engine optimization, property-based testing